

Technischer Pentest-Abschlussbericht & Remediation-Report

Öffentliche Portfolio-Version - sensible Details entfernt

Hinweis: Dieser Bericht wurde mit **ausdrücklicher Autorisierung des Betreibers** erstellt. Dies ist eine bereinigte öffentliche Fassung; Domain-Name, Benutzernamen, IP-Adressen und spezifische Infrastrukturdetails wurden zum Schutz des Ziels entfernt oder durch Platzhalter ersetzt.

Zielobjekt	[REDACTED] (autorisierte Live-WordPress-Domain)
Untersuchungsdatum	Juni 2026
Methodik	Blackbox External Testing (kein Zugriff auf Origin-Server)
Audit-Plattform	Intel NUC 15 Pro, Arch Linux (KVM/QEMU + Kali Linux VM)
Scope	Öffentliche Schnittstellen der Ziel-Domain
Out of Scope	Origin-Server-IP, interne Netzwerke, Social Engineering
Status	Abgeschlossen- alle Restrisiken behoben

1. Executive Summary

Ergebnis: Es wurden keine kritischen, akut von außen ausnutzbaren Sicherheitslücken identifiziert. Sämtliche festgestellten Abweichungen von Best-Practices wurden anschließend gemeinsam mit dem Betreiber behoben.

Die öffentlich erreichbaren Schnittstellen und Dienste der autorisierten Ziel-Domain wurden einer externen Sicherheitsüberprüfung im Blackbox-Verfahren unterzogen. Der Fokus lag auf offenen Ports, der TLS-Konfiguration, fehlenden Security-Headern sowie der Anwendungssicherheit der WordPress-Kommentarfunktion.

Die Infrastruktur ist durch einen vorgeschalteten Reverse-Proxy (Cloudflare) sowie serverseitige Schutz-Plugins gegen automatisierte Angriffe, SSL-Schwachstellen und Missbrauch abgesichert.

Befund-Übersicht

#	Titel	Risiko	Status
1	TLS-Konfiguration & Krypto-Analyse	Niedrig	Kein Handlungsbedarf
2	Fehlender Brute-Force-Schutz (Rate Limiting)	Mittel	Behoben
3	Username Enumeration via REST API	Mittel	Empfohlen (offen)
4	Fehlende HTTP-Sicherheits-Header	Mittel	Behoben
5	HTML-Injektion im Kommentarfeld	Niedrig	Akzeptiertes Restrisiko
6	Fehlende Anti-CSRF-Tokens	Niedrig	Infrastruktur abgesichert
7	Offene alternative Web-Ports	Informativ	Kein Handlungsbedarf
8	Slowloris DoS (CVE-2007-6750)	Niedrig	Behoben

2. Netzwerkanalyse & Port-Scanning (Reconnaissance)

Ein vollständiger TCP-Port-Scan mittels **Nmap** identifizierte die aktiven Schnittstellen am Perimeter. Sämtliche Web-Ports werden über einen Cloudflare-Reverse-Proxy bereitgestellt; HTTP wird konsequent auf HTTPS umgeleitet. Ein administrativer Management-Port (WHM/cPanel) war erreichbar- die konkreten Portnummern und Adressdetails sind in dieser öffentlichen Fassung entfernt.

Eingesetzte Tools: Nmap (Port- und Service-Enumeration, *--script vuln*)

3. Detaillierte Befunde & Technische Analyse

Befund 1: TLS-Konfiguration & Krypto-Analyse

Risiko: **Niedrig**

Beschreibung: Analyse der TLS-Konfiguration des Perimeters auf veraltete Protokolle und bekannte Krypto-Schwachstellen.

Nachweis: `sslyze -regular [TARGET]`

Der Server akzeptiert ausschließlich TLS 1.3 und lehnt alle veralteten Protokolle (SSL 2.0/3.0, TLS 1.0/1.1) konsequent ab. Nachweislich geschützt gegen OpenSSL CCS Injection (CVE-2014-0224), Heartbleed, ROBOT-Attack sowie Downgrade-Angriffe (TLS_FALLBACK_SCSV).

Bewertung: Eine formale Abweichung von der Mozilla-'Intermediate'-Konfiguration (Unterstützung von secp521r1) stellt im vorliegenden Szenario keine ausnutzbare Lücke dar.

Status / Empfehlung: Kein Handlungsbedarf.

Befund 2: Fehlender Brute-Force-Schutz (Rate Limiting)

Risiko: **Mittel**

Beschreibung: Prüfung der Authentifizierungs-Endpunkte auf Schutzmechanismen gegen automatisierte Passwort-Angriffe.

Nachweis: `wpscan --url [TARGET] --enumerate vp,vt,u --api-token [REDACTED]`

Der initiale Scan mit WPScan verlief ohne Blockierung durch eine WAF oder ein IPS. Es konnten valide Systembenutzer extrahiert werden (Benutzernamen in dieser Fassung entfernt). Die Authentifizierungs-Endpunkte (wp-login.php, xmlrpc.php) wiesen im Ursprungszustand keine Schutzmechanismen gegen automatisierte Passwort-Angriffe auf. Ein Testlauf bestätigte, dass hunderte Brute-Force-Anfragen pro Minute ohne IP-Sperrung oder Drosselung verarbeitet wurden.

Bewertung: In Kombination mit der Username Enumeration (Befund 3) ergibt sich ein konkreter Angriffspfad.

Status / Empfehlung: Behoben (siehe Abschnitt 5.1).

Befund 3: Username Enumeration via WordPress REST API

Risiko: **Mittel**

Beschreibung: Prüfung auf anonyme Preisgabe interner Benutzernamen über die WordPress-REST-API.

Der anonyme Zugriff auf den REST-API-Endpunkt `/wp-json/wp/v2/users` gibt intern registrierte Benutzernamen preis. Diese lieferten die Basis für die Brute-Force-Versuche aus Befund 2.

Status / Empfehlung: Anonymen Zugriff auf den Endpunkt via `functions.php` oder Sicherheits-Plugin (z. B. Wordfence) blockieren. Zum Berichtszeitpunkt offen.

Befund 4: Fehlende HTTP-Sicherheits-Header

Risiko: **Mittel**

Beschreibung: Passive Analyse der HTTP-Response-Header auf clientseitiges Browser-Policy-Enforcement.

Nachweis: Burp Suite - Passive HTTP Response Header Analyse

Es fehlten: *HSTS* (kein HTTPS-Enforcement, SSL-Stripping-Risiko), *CSP* (kein Schutz gegen Script-Injection), *Referrer-Policy*, *X-Content-Type-Options* (MIME-Sniffing) und *X-Frame-Options* (Clickjacking).

Bewertung: Hinweis zu X-XSS-Protection: bewusst nicht nachgerüstet- der Header ist veraltet, von Chrome/Edge entfernt, von Firefox nie implementiert und kann neue Lücken einführen. Korrekte Lösung ist eine CSP.

Status / Empfehlung: Behoben (siehe Abschnitt 5.2).

Befund 5: HTML-Injektion mit Inline-Style-Attributen (Kommentarfeld)

Risiko: **Niedrig**

Beschreibung: Prüfung der WordPress-Kommentarfunktion auf unzureichende Eingabefilterung.

Das Kommentarfeld filtert Eingaben unvollständig; einfache HTML-Tags und Inline-Style-Attribute werden gerendert. Dies erlaubt visuelle Manipulation, jedoch keine Codeausführung. Terminologie-Präzisierung: Korrekt handelt es sich um HTML-Injektion mit Inline-Style-Attributen, nicht um 'CSS Injection' (= Datenexfiltration via CSS-Selektoren).

Bewertung: Kompensierende Kontrollen (wp_kses): Aktiver JavaScript-Schadcode (Stored XSS via <script>, Event-Handler) wird zuverlässig gefiltert; Phishing-Formulare blockiert; Moderationsschleife verhindert öffentliche Sichtbarkeit ohne Admin-Freigabe.

Status / Empfehlung: Restrisiko akzeptiert.

Befund 6: Fehlende Anti-CSRF-Tokens

Risiko: **Niedrig**

Beschreibung: Prüfung der Kommentar-POST-Anfrage auf CSRF-Schutz.

Die POST-Anfrage an wp-comments-post.php enthält kein _wpnonce-Token. WordPress erzwingt dies für anonyme Kommentare standardmäßig nicht- isoliert betrachtet eine CSRF-/Spam-Anfälligkeit.

Bewertung: Kompensierende Schutzmechanismen (bestätigt via nmap --script vuln): Cloudflare WAF (Browser-Validierung via Clearance-Cookie) und Antispam Bee (dynamischer Honeypot mit vertauschten Eingabefeldern) neutralisieren automatisierte Angriffe zuverlässig.

Status / Empfehlung: Durch Infrastruktur abgesichert.

Befund 7: Offene alternative Web-Ports

Risiko: *Informativ*

Beschreibung: Service-Erkennung auf den alternativen Web-Ports.

Hinter den alternativen Web-Ports sind keine lokalen Serverdienste offengelegt; sie werden vollständig von Cloudflare-Gateways verwaltet (HTTP-301-Redirect bzw. reguläres SSL-Zertifikat).

Status / Empfehlung: Kein Handlungsbedarf.

Befund 8: HTTP-Slowloris DoS-Anfälligkeit (CVE-2007-6750)

Risiko: **Niedrig**

Beschreibung: Prüfung des Webserver auf Anfälligkeit gegen Slowloris (Thread Starvation durch langsam gesendete, unvollständige HTTP-Header).

Der thread-basierte Webserver ist grundsätzlich anfällig gegenüber Slowloris.

Bewertung: Korrigierte Einstufung: Da Cloudflare als Reverse-Proxy alle TCP-Verbindungen terminiert, erreicht ein Slowloris-Angriff den Origin-Server unter Normalbedingungen nicht. Einstufung von Mittel auf Niedrig korrigiert. Eine serverseitige Härtung wurde dennoch zur Tiefenverteidigung umgesetzt.

Status / Empfehlung: Behoben (siehe Abschnitt 5.3).

4. Bedrohungsmodellierung & Angriffs-Verkettung

Zur Illustration des theoretischen Maximalrisikos wurde eine Angriffskette aus den Befunden 2, 4 und 5 evaluiert. Wichtig: Die aktiven Schutzmechanismen (Cloudflare WAF, Honeypot, wp_kses) machen eine vollständige Ausnutzung im Ist-Zustand unwahrscheinlich. Die Kette dient ausschließlich der konzeptionellen Darstellung.

Der Vektor erfordert eine Benutzerinteraktion des Administrators (Klick auf einen präparierten Link). Ein verstecktes, automatisiert absendendes Formular nutzt das fehlende Anti-CSRF-Token aus. Der konkrete Payload ist in dieser öffentlichen Fassung entfernt; er war auf HTML-Injektion (visuelle Manipulation) beschränkt, da wp_kses aktive Script-Handler zuverlässig filtert. Ein Stored XSS im Admin-Kontext ist im Ist-Zustand nicht realisierbar.

5. Sicherheits-Remediation & Verifikation

Sämtliche Defizite wurden gemeinsam mit dem Betreiber behoben. Der Brute-Force-Schutz wurde nach der Remediation erneut getestet und verifiziert (HTTP 403); die übrigen Maßnahmen wurden umgesetzt.

5.1 Brute-Force-Schutz & WAF

Serverseitiges Rate Limiting und aktiver WAF-Regelsatz. Verifikation: erneuter WPScan-Versuch wird mit HTTP 403 Forbidden abgewiesen.

5.2 HTTP-Sicherheits-Header

HSTS, Content-Security-Policy, Referrer-Policy, X-Content-Type-Options und X-Frame-Options wurden restriktiv nachgerüstet (X-XSS-Protection bewusst ausgelassen, ersetzt durch CSP).

5.3 Slowloris-Mitigation (Defense in Depth) mod_reqtimeout mit Header- und Body-Timeouts konfiguriert; künstlich verzögerte Sockets werden serverseitig getrennt.

5.4 Perimeter & SameSite-Cookies

Host-Firewall auf die spezifischen Cloudflare-Proxy-Ports beschränkt (Details entfernt), alle weiteren Ports geblockt. Session-Cookies mit SameSite=Lax versehen.

6. Strategische Restempfehlungen

- **Username Enumeration blockieren:** anonymen Zugriff auf /wp-json/wp/v2/users via functions.php oder Plugin unterbinden.
- **IP-Whitelisting für den Management-Port:** WHM/cPanel-Zugang auf feste Admin-IPs beschränken.
- **CSP-Verschärfung:** Policy schrittweise auf tatsächlich genutzte Ressourcen zuschneiden (CSP-Evaluator).
- **Regelmäßige Updates:** WordPress-Core, Plugins und Themes aktuell halten; periodische WPScan-Läufe.

Ende des Berichts - Öffentliche Portfolio-Fassung. Durchgeführt mit Autorisierung des Betreibers.